Fichamento: A systematic comparison of LLM performance for intrusion detection

1. Referência Bibliográfica: BUI, Minh-Thanh et al. A Systematic Comparison of Large Language Models Performance for Intrusion Detection. **Proc. ACM Netw.**, v. 2, n. CoNEXT4, Article 22, Dezembro 2024.

2. Tema e Objetivo: O estudo investiga a viabilidade e a eficácia da aplicação de *Large Language Models* (LLMs) para a detecção de intrusões, focando na identificação de padrões maliciosos diretamente em pacotes de rede brutos (*raw packets*) e logs gerados por *firewalls* comerciais. O objetivo central é avaliar se as tecnologias baseadas em *Transformers* (desde modelos pequenos até fundacionais) conseguem substituir ou auxiliar sistemas tradicionais de detecção (IDS) e analistas humanos na classificação da severidade de ataques e na geração de relatórios de incidentes em linguagem natural.

3. Metodologia e Abordagem Experimental: Os autores estruturaram o estudo para evitar armadilhas comuns em pesquisas de segurança com Machine Learning (como viés de amostragem e uso apenas de dados de laboratório).

- **Base de Dados (Mundo Real):** Utilizou-se um *dataset* proprietário coletado ao longo de 5 meses, contendo mais de 2,06 milhões de eventos de segurança reais com *payloads* não vazios. Os dados incluem metadados (tuplas de 5, portas, protocolos) e *logs* de alarmes classificados em 5 níveis de severidade (de 1: Ataque bem-sucedido a 5: Tráfego não especificado). O *dataset* apresentou forte pluralidade, com 2.500 tipos únicos de ataques.

- **Cenários de Avaliação:**
    1. **Seleção Estratificada:** Usada para balancear as classes no treinamento e isolar a capacidade de aprendizado dos modelos, dividindo os dados em 80/20.
    2. **Divisões Temporais (*Time-based splits*):** Para simular um ambiente de produção sujeito a *data drift* e ataques *zero-day*, os dados foram divididos em cortes temporais (ex: dados até 15 de agosto para treino, resto para teste).

- **Técnicas Contrastadas:**
    1. **Engenharia de Prompt (*Few-shot*):** Avaliação de LLMs congelados comerciais (GPT-3.5, GPT-4) e *open-source* locais (Mistral-7B, Llama2-13B) recebendo o *payload* bruto para classificar a ameaça.
    2. **RAG (*Retrieval-Augmented Generation*):** Criação de um "AttackDB" (usando ChromaDB e Langchain) para vetorizar *payloads* históricos. Novos pacotes buscam os top-k *payloads* mais similares para auxiliar o LLM na classificação e explicação.
    3. **Fine-tuning de Encoders:** Ajuste fino de modelos menores baseados em *Transformers* (BERT, BigBird, UniXcoder, SecureBERT, GPT-2) usando

representações de logs para prever a severidade da ameaça. Estes foram comparados a uma forte linha de base de mais de 50 modelos clássicos de ML baseados em TF-IDF.

4. Principais Resultados e Descobertas

- **Limitações Críticas do *Prompting* Direto:** Modelos de linguagem, mesmo os mais avançados como o GPT-4, apresentaram desempenho insatisfatório ao processar *logs* e *payloads* crus (F1-score em torno de 0.28 a 0.46). Houve uma tendência alarmante de classificar alarmes falsos como ataques bem-sucedidos.

- **O RAG como Ponte, mas não Solução Definitiva:** O RAG melhorou significativamente a acurácia na recuperação do nome do evento (*Recall@1* variando de 0.56 a 0.79 dependendo do cenário temporal), mas, isoladamente, a similaridade de cosseno nos vetores de *payload* não foi suficiente para substituir a classificação de um IDS formal.

- **A Supremacia do *Fine-Tuning* sobre o ML Tradicional:** Modelos LLM ajustados (*fine-tuned*), particularmente o BERT (110M parâmetros), superaram consistentemente a linha de base de ML. Ao fundir dados de metadados (tupla-5) e *payload*, o BERT alcançou mais de 95% de acurácia ponderada para ataques já conhecidos.

- **A Relação Custo-Benefício do Tamanho do Modelo:** A pesquisa demonstrou que treinar modelos massivos para essa tarefa específica é um desperdício de recursos. Fazer *fine-tuning* em um modelo de 7 Bilhões de parâmetros (Mistral) não trouxe ganhos de performance em relação a um BERT clássico de 110 Milhões de parâmetros. Além disso, mecanismos de atenção esparsa para janelas de contexto maiores (como os 4096 tokens do BigBird) degradaram a performance, indicando que a assinatura maliciosa do log geralmente reside no início do pacote.

- **A Barreira do *Zero-Day*:** Todos os modelos sofreram quedas agudas (caindo para cerca de 70% de acurácia) ao serem testados nos cortes temporais que incluíam ataques inéditos (*out-of-distribution*), evidenciando que a capacidade de generalização de LLMs para novas ameaças de rede ainda é muito limitada.

5. Implicações Práticas para Sistemas de Análise de Logs e Cibersegurança O artigo propõe um modelo operacional *desacoplado* altamente eficiente para integração em *Pipelines* de Segurança (SOCs):

- **Pipeline de Duas Etapas:** Em vez de usar um LLM gigante para tudo, a arquitetura ideal utiliza um modelo *Encoder* pequeno e rápido (ex: BERT com *fine-tuning*) operando em linha para analisar a volumetria de *logs* brutos e classificar o risco. Apenas quando o risco é alto, um LLM fundacional (LLMSec-Assistant) é acionado para ler esses resultados e redigir o relatório.

- **Explicabilidade de Logs (*Explainability*):** Para contornar a natureza "caixa-preta" das redes neurais, o estudo usa métodos como *Integrated Gradients* (via framework Captum) para medir a importância individual de cada *token* no log e o *KeyBERT* para extrair as palavras-chave críticas do *payload*. Esses *tokens* destacados alimentam o LLM maior, garantindo que o relatório textual em linguagem natural gerado para o analista seja fundamentado na evidência real do ataque.

- **Gestão de CTI (Cyber Threat Intelligence):** O sistema final agrupa o *payload* bruto, as variáveis do RAG, os *tokens* atencionais do modelo menor e dados de CTI (reputação de IPs) no *prompt* do LLM congelado, fornecendo um diagnóstico completo e inteligível para o operador humano.